University of Hertfordshire UH

School of Physics, Engineering and Computer Science

Assignment Briefing Sheet (2026/27 Academic Year)

Section A: Assignment title, important dates and weighting

Assignment title:	Assignment 1 –AstroBot	Group or individual:	Individual
Module title:	Web Technology	Module code:	5FTC2167
Module leader:	**Meena Mengle/ Robert T**	Moderator's initials:	
Submission deadline:	**TBC**	Target date for return of marked assignment:	

You are expected to spend about 40 hours to complete this assignment to a satisfactory standard.

This assignment is worth **50%** of the overall assessment for this module.

Section B: Student(s) to complete

Student ID number	Year Code

Notes for students

- For undergraduate modules, a score above 40% represent a pass performance at honours level.
- For postgraduate modules, a score of 50% or above represents a pass mark.
- Late submission of any item of coursework for each day or part thereof (or for hard copy submission only, working day or part thereof) for up to five days after the published deadline, coursework relating to modules at Levels 0, 4, 5, 6 submitted late (including deferred coursework, but with the exception of referred coursework), will have the numeric grade reduced by 10 grade points until or unless the numeric grade reaches or is 40. Where the numeric grade awarded for the assessment is less than 40, no lateness penalty will be applied.
- Late submission of referred coursework will automatically be awarded a grade of zero (0).
- Coursework (including deferred coursework) submitted later than five days (five working days in the case of hard copy submission) after the published deadline will be awarded a grade of zero (0).
- Regulations governing assessment offences including Plagiarism and Collusion are available from https://www.herts.ac.uk/about-us/governance/university-policies-and-regulations-uprs/uprs (please refer to UPR AS14)
- Guidance on avoiding plagiarism can be found here: https://herts.instructure.com/courses/61421/pages/referencing-avoiding-plagiarism?

module_item_id=779436

- Modules may have several components of assessment and may require a pass in all elements. For further details, please consult the relevant Module Handbook (available on Studynet/Canvas, under Module Information) or ask the Module Leader.

Assignment Briefing Sheet (2025/26 Academic Year)

This Assignment assesses the following module Learning Outcomes (from Definitive Module Document):

KU1: Be able to explain the various components of web architecture, and common design patterns.

IPT1:Be able to integrate API calls from different services using appropriate programming language, by identifying and utilizing appropriate HTTP response codes, requests, and API documentation.

IPT2: Be able to develop and parse common data formats (e.g., XML, JSON, YAML) into data structures for programming languages such as Python or JavaScript.

Assignment Brief:

Astro Bot for the Astronomy Club

The Astronomy Club is a student-led group that organises stargazing events, ISS observation sessions, and outdoor astronomy activities. Club members use the **Cisco Webex messaging application** to communicate, share updates, and coordinate events.

As part of a wider initiative to **revamp their IT network infrastructure** (explored further in Assignment 2), members can now access Webex on their mobile devices via Wi-Fi. To further enhance their digital capabilities, the club plans to deploy an **Bot** on one of their servers to automate information sharing and support event planning.

You are required to develop a **Python-based Astro Bot** (using the provided starter code) for the Astronomy Club. The bot will support planning and decision-making by integrating data from multiple web-based services (APIs).

The Astro Bot will connect to a **Webex messaging space**, where users can send commands such as /weather cityname and /iss to request real-time information. The bot will monitor the selected Webex room, process these commands, retrieve relevant data from external APIs, and return a **clear, well-formatted, and user-friendly response** back into the chat.

You are required to use GitHub as part of your development process. This includes storing your code in an online repository, regularly tracking your progress through meaningful commits, and maintaining version control throughout the project. This will demonstrate your development workflow and allow you to manage changes effectively as your AstroBot evolves.

You must integrate the following APIs into your solution:

1. **Messaging API (Webex)**
   Used to monitor messages and send responses in a Webex room
   https://developer.webex.com/messaging/docs/getting-started
2. **Weather API (OpenWeather)**
   Used to retrieve weather conditions for astronomy event planning
   https://openweathermap.org/guide#how_to_start
3. **ISS Location API**
   Used to track the real-time position of the International Space Station

http://open-notify.org/Open-Notify-API/ISS-Location-Now/

4. **Geocoding / Reverse Geocoding API**
   Used to convert latitude and longitude into a readable location or vice versa (e.g., city, country)
   https://openweathermap.org/api/geocoding-api?collection=other

Resources Provided

You will be given the following resources to support your development:

- **API Investigation Documentation Sheet** (README_template.md)
  To record your research on each API
- **Python Starter Script** (astro_bot.py)
  Contains partially completed code with sections for you to complete

IT IS MANDATORY TO USE THE RESOURCE PROVIDED

NOTE: To protect application environments such as Webex from bots or malicious access attempts, most APIs apply rate limiting. If you make a large number of repeated API calls in a short period, your access may be temporarily blocked.

This timeout is usually less than 5 minutes, but you should check the specific API documentation for details.

Task 1: API Investigation (40 Marks)

Save and complete the provided README.md investigation template in your GitHub repository, ensuring all required findings are clearly documented as outlined in the file.

- Explore the documentation for the Webex Messaging API, Weather API, ISS Location API, and a Geocoding API to identify key technical details such as endpoints, authentication methods, and JSON response formats. Tools such as Postman may be used to test API requests and examine responses. (Complete before Task 2)
- Research how Python handles time conversion (epoch to human-readable format), as this will be required when processing API data. (Complete during Task 2)
- Demonstrate understanding of key web concepts, including client–server architecture, RESTful APIs (e.g., GET and POST methods), and the MVC design pattern. (Complete after Task 2)
- Log key technical details such as endpoints, authentication methods and JSON response formats for any additional APIs used as part of the extension task.

Record all findings in the README.md file within the GitHub repository, ensuring it is kept up to date.

Task 2: Develop the Astro Bot (30+15 Marks)

In this task, the Astro Bot will be developed by completing the partially provided Python starter code. This task builds directly on the API knowledge gained in Task 1.

The starter code must be developed step-by-step, with regular testing to ensure functionality at each stage. Progress should be clearly evidenced by GitHub commits, with meaningful messages (refer to Conventional Commits [https://www.conventionalcommits.org/en/v1.0.0/]) to demonstrate development over time for

each step.

Coding Strategy

The starter code is structured into functions and modules. You are required to complete the missing sections while maintaining this modular design. Further step by step instructions can be found in the appendix A of this assignment brief.

Extended Feature – Astronomy Enhancement

Extend the Astro Bot to support additional functionality for the Astronomy Club by integrating at least one astronomy or stargazing-related API. This should enhance the usefulness of the bot for planning observation events and improving the user experience.

Research and select an appropriate API and implement it within the existing modular structure. The new feature must be triggered using a new command (e.g., /stars, /moon, /sky) and follow the same approach used in Task 2, including API calls, data processing, and user-friendly output formatting.

Suggested APIs for Extension

- AstronomyAPI → Provides data on planets, stars, and moon phases
- NASA Open APIs → Astronomy Picture of the Day, space events and imagery
- Open-Meteo (Astronomy/Cloud data) → Useful for stargazing conditions (cloud cover, visibility)
- WeatherAPI Astronomy endpoint → Sunrise, sunset, and moon phase data

NOTE: In order to achieve higher grade 70% + in this Task you need to demonstrate good coding practice including appropriate use of comments. Effective error handling should be implemented to manage issues such as invalid API responses, missing data, and incorrect user input. The solution should also provide a good user experience through clear, well-formatted outputs. Consideration of security is important, including safe handling of API keys, awareness of rate limits, and input validation, resulting in a more robust and professional solution.

Task 3: Version Control (5 marks) and Video (Including Audio) Demonstration (10 Marks)

Record a short video (3 minutes) with audio to demonstrate the main functionalities of your Astro bot and extended features. Narrative should also include justification on choice of API and security issues.

Appropriate screen recording software should be used to record this video, poor quality of audio and video (e.g., recorded from mobile phone) will affect your marks for this task.

Use Git for version control. Maintain a repository on GitHub to track your project's progress and changes. Ensure regular commits for each step with meaningful messages that document your development process.

Submission Requirements:

All submission should be made via Canvas, namely:

- All submissions should be made via Canvas and must include:
- a link to your GitHub repository, including the completed README.md file and astro_bot.py file;
- a short video demonstration with audio showing the main functionality of the Astro Bot and the extended feature.

Marks awarded for:

Components	Marks
Task 1: Astro Bot API Investigation *Webex Messaging API* — *4%* *ISS Current Location API* — *2%* *Geocoding API* — *4%* *Weather API* — *3%* *Epoch to Human Time Conversion* — *2%* *Extended feature API* — *3%* *Web Architecture* — *6%* *RESTful API* — *5%* *MVC Design Pattern* — *3%* *Reflection* — *8%*	40%
Task 2: Functionality and Requirement Programming Bot *Functionality & Requirements* — *6%* *Bot correctly monitors Webex room, processes /iss and /weather commands, retrieves API data, and sends responses. Messages are processed correctly and duplicates are avoided.* *API Integration* — *12%* *Correct use of APIs (Webex, ISS, Weather, Geocoding). Proper use of endpoints, parameters, and HTTP requests/responses.* *Data Handling & Processing* — *6%* *JSON data is correctly parsed and used. Timestamp conversion and data extraction are handled appropriately.* *Output Formatting & User Experience* — *6%* *Output is clear, readable, and user-friendly. Handles missing data appropriately (e.g., N/A, Unknown).*	30%
Extended Feature *Extension Functionality* — *6* *New feature implemented using an additional astronomy/stargazing API. Triggered via a new command (e.g., /stars, /moon).* *Integration & Application* — *4* *Correct integration of new API into existing code structure. Uses appropriate API endpoints and data*	15%

handling.		
Enhancement Quality *Feature adds clear value for the Astronomy Club (e.g., stargazing conditions, moon phase, sky visibility).*	*3*	
Code Quality & Modularity *Extension is implemented as a separate function, maintaining modular design and readability.*	*2*	
Task 3: Version Control and Video (Including Audio) Demonstration		15%
Version Control: Regular commits with meaningful description showing progression	*5%*	
Video Demonstration: narrative discussing the choice of API's and security issues, while demonstrating the working of the Astro Bot clear audio and video	*10%*	
Total		**100%**

Type of Feedback to be given for this assignment:

Each individual student will receive written feedback about this assignment, as well as marks awarded based on the criteria set in the next section.

If the assignment is laboratory based (though not computer-based), or involves offsite activity, please attach the risk assessment form for the Internal Moderator to see.

Marks	Investigation of API (40%)	Functionality and Requirement Programming Bot (30%)	Extended Feature (15%)	Version control and video demonstration (15%)
90 – 100 (outstanding)	Outstanding... Level of detailed investigation and fully correct and well formatted Readme.md	Outstanding Fully functional and robust Bot with excellent modular code structure, clear comments, and accurate API integration. Strong error handling, secure handling of API keys, and highly user-friendly output. May include enhancements beyond requirements.	Highly relevant and well-integrated extension using an additional API. Fully functional, modular, and enhances the Bot significantly. Components are fully functional as required and occasionally stepping beyond expectations using sophisticated solutions.	Outstanding. Level of narrative discussing the choice of API and security issues. Professional audio and video Regular and short commits with meaningful description showing progression for each step.
80 – 89 (Excellent)	Excellent... Level of detailed investigation of API, with minor discrepancies and/or missing items in Readme.md	Excellent Fully functional solution with well-structured code and effective API use. Minor errors do not affect overall performance. Good commenting, error handling, and clear output.	Excellent use of appropriate technology as applied to the problem domain. Well-implemented extension with effective API integration. Functional and adds clear value and with a generally excellent and highly accurate application of skills and techniques.	Excellent level of narrative and justification of API choices and security with minor omissions. Professional audio and video. Regular and short commits with some description showing progression for each step
70 – 79 (Very good)	Very good... Level of detail with few discrepancies and/or missing items in Readme.md	Very good Functional solution meeting most requirements with good structure and modular design. Some error handling and clear output present, with minor issues.	Very good use of appropriate technology as applied to the problem domain. The extension is functional and demonstrates very good understanding. A new API is used appropriately, and the feature is implemented with reasonable structure. Some minor issues may exist in integration or output formatting, but with a high level and accurate application of skills and techniques.	Very good. level of narrative and justification of API choices and security with small omissions. Professional audio and video. Regular and short commits with some description showing progression.
50 – 69 (Clear Pass/Good)	Good... Level of detail with some discrepancies and/or missing items in Readme.md .	Meets core requirements with basic API integration. Code structure is present but inconsistent. Limited error handling and average output quality. Good use of techniques to derive solutions.	Good use of appropriate technology as applied to the problem domain. Good and reasonably accurate application of skills and techniques demonstrated. Some errors in technique and/or application with minor impact on deliverables; required components may not be fully functional but may be easily resolved.	Good narrative but rough audio video. Good discussion of API, and some security issues. Short commits but shows progression.
40 – 49 (Satisfactory)	Good Level of detail with quite a few discrepancies and/or missing items in Readme.md	Partially functional solution with gaps in API use, structure, or output. Limited modularity and minimal error handling. Techniques are applied to derive solutions but steps are missed.	Satisfactory use of appropriate technology as applied to the problem domain. Satisfactory application of skills and techniques demonstrated but with minor inaccuracies. Errors in technique and/or application with some impact on deliverables; required components may suffer from notable errors that affect overall functionality, but which can be rectified by minor improvement.	Potentially vague and unclear and/or rough audio video. Discussion satisfactory but with minor inaccuracies or rushed, lacks details. Some commits but shows progression.
30 – 39 (Marginal fail)	Poor... Level of detail with many discrepancies and/or missing items, no readme.md or other format submitted.	Limited functionality with major errors in API integration and code structure. Output unreliable or incomplete.Limited use of techniques to derive solutions.	Limited use of appropriate technology as applied to the problem domain. Limited application of skills and techniques demonstrated. Many errors in technique and/or	Poor audio video but has some demonstration value. Few large commits, in short span of time description included but does not show progression.

			application with high impact on deliverables; required components may suffer from critical errors and may require significant improvement.	
20 – 29 (Clear fail)	Weak... Level of detail with too many discrepancies and/or missing items, no readme.md or other format submitted Certain elements of academic misconduct.	Minimal functionality with significant errors. Code lacks structure and correct API use. Little use of techniques to derive solutions. Certain elements of academic misconduct.	Very little use of appropriate technology as applied to the problem domain. Very little skill and application of techniques demonstrated. High number of errors with very high impact on deliverables; required components may not function in any discernable way and/or are entirely missing. Certain elements of academic misconduct.	Video extremely poor lacking either voice over or graphics. Few large commits, in short span of time description missing or unclear. Certain elements of academic misconduct.
0 – 19 (Fail – nothing of merit)	Inadequate... Level of detail that is muddled and/or inconsistent, no readme.md Academic misconduct.	No valid or functional solution provided. No application of techniques to derive solutions. Academic misconduct.	No use of appropriate technology as applied to the problem domain. No skill and application of technique demonstrated. Very high number of errors in deliverable or no deliverable submitted. Required components are under developed, missing, or inappropriate. Academic misconduct.	No video or corrupt file, unable to open. Poor quality no clarity. No GitHub link or not accessible, missing commits or one large commit. Academic misconduct.

APPENDIX A: STEP BT STEP INSTRUCTIONS FOR COMPLETING TASK 2

Step 1: Import Required Libraries

Import all necessary libraries for:

- API requests (requests)
- JSON processing (json)
- Time handling (time)
- *For higher grade*, consider importing env file parsing library (python-dotenv)

Step 2: Configure API Credentials

Complete the functions to:

- Ask the user to **use a hardcoded Webex token or enter their own**
- Ask the user to **use a hardcoded Weather API key or enter their own**
- *For higher grade*, consider loading hardcoded keys from an env file for security.

Step 3: Retrieve Webex Rooms

- Provide the correct **Webex Rooms API URL**
- Send a GET request using the access token
- Store and process the JSON response

Step 4: Display Available Rooms

- Complete the loop to display:
    - Room name
    - Room type
- Ensure output is clear and readable

Step 5: Select a Room to Monitor

- Prompt the user to choose a room
- Match the user input with available rooms
- Store the selected **room ID**

Step 6: Monitor Messages in the Room

- Provide the correct **Webex Messages API endpoint**
- Poll the room every **5 seconds**
- Retrieve the latest message

The bot must:

- Only process **new messages**
- Ignore duplicate messages
- Ignore messages that do not begin with /

Step 7: Process User Commands

The bot must respond to the following commands:

/iss Command

- Call the ISS API to retrieve:
    - Latitude
    - Longitude
    - Timestamp
- Convert timestamp into a **human-readable date/time**

- Use the Geocoding API to convert coordinates into a location
- Use the Weather API to retrieve conditions at that location

/weather cityname Command

- Extract the entered city name
- Retrieve current weather data such as:
    - Description
    - Temperature
- Determine whether conditions are suitable for stargazing

Step 8: Handle API Data

- Extract required values from JSON responses
- Ensure missing data is handled safely (e.g., N/A, Unknown)

Step 9: Format the Output

- Complete the formatting functions to generate clear output messages
- Ensure responses include:
    - Date and time
    - Location
    - Coordinates (for ISS)
    - Weather information

Output must be **user-friendly and well-structured**

Step 10: Send Response to Webex

- Provide the correct **Webex POST API endpoint**
- Construct the request body
- Send the formatted message back to the selected room

Step 11: Testing the Bot

1. Create a Webex room (e.g., *Astro Club Room*)
2. Add initial messages (e.g., “Hello”)
3. Run the bot
4. Select the room
5. Enter commands in Webex:

/iss

/weather cityname

1. Verify that:

- The bot responds correctly
- Messages are not repeated
- Output is clear